

CS224R Final Project Extended Abstract

Librarian Models: Anticipatory Filesystem Construction via Reinforcement Learning

Teresa Zhang
Stanford University
teresaz@stanford.edu

Today, large language models are increasingly expected to answer questions over document collections that are too large, fragmented, or heterogeneous to fit into context at once, yet standard retrieval-augmented generation systems remain largely reactive: after each query arrives, they search the raw corpus, retrieve passages, and synthesize an answer, repeatedly rediscovering cross-document relationships that could instead be organized once and reused across future queries. In this project, we ask whether an agent can restructure a corpus before queries arrive, constructing a reusable memory artifact that shifts part of long-context reasoning from query time to an amortized precomputation phase.

Librarian Models instantiates this idea through an anticipatory memory framework in which a builder agent transforms an unstructured corpus into an augmented filesystem of synthetic files, summaries, directories, and provenance links, arranged around operations that language agents already perform well: scanning filenames, reading concise artifacts, traversing hierarchy, and falling back to source documents when synthesized memory is insufficient. Rather than relying on a rigid knowledge graph or query-specific cache, the goal is for the builder to create an answer-facing workspace intended to support many plausible future questions while preserving raw-document access for verification.

The central contribution of this project is to formulate filesystem construction as a finite-horizon Markov Decision Process and optimize the builder with reinforcement learning. The state is the current filesystem, initially containing only raw documents; the action space consists of constrained edits, including creating synthetic files, merging related artifacts, deleting redundant synthetic files, and stopping construction; and the reward evaluates whether the resulting filesystem improves downstream question-answering accuracy for a frozen answering agent while penalizing construction cost. To improve credit assignment, we also use a dense marginal reward measuring the change in expected answering performance after each edit, and we optimize the builder policy with GRPO while holding the answering agent fixed, so that gains can be attributed to learned memory construction rather than to a stronger answerer.

Empirically, Librarian Models improves long-context document reasoning on the BrowseComp-Plus dataset and transfers to out-of-distribution BRIGHT domains. On BrowseComp-Plus, the RL-trained builder outperforms reactive retrieval, reranking, GraphRAG-style precomputation, agentic RAG, auto-compaction, and a non-finetuned builder ablation, showing that RL is the mechanism that teaches the builder to create navigable, reusable memory structures. For example, the non-finetuned builder often produces fragmented filesystems that are difficult for the answering agent to use, whereas the RL-trained builder constructs broader and deeper synthetic structures that make relevant information easier to find, and amortization experiments show that upfront construction cost can be spread across repeated queries over the same document set. On BRIGHT, where the builder is evaluated zero-shot on StackExchange, coding, and theorem-proving domains outside its training distribution, Librarian Models continues to outperform the base builder, suggesting transferable organization heuristic.

Overall, the main lesson is that memory construction can be treated as an RL problem rather than as a fixed preprocessing heuristic: by optimizing a builder agent directly for downstream answerability, Librarian Models trains an agent not only to act in an external environment, but also to construct the information environment that future agents will navigate. However, the project also exposes limitations, since synthesized memories can omit fine-grained evidence, hallucinate or compress away crucial details, and become difficult to maintain in dynamic corpora, which is why provenance links and fallback access to raw documents remain essential.

Librarian Models: Anticipatory Filesystem Construction via Reinforcement Learning

Anikait Singh Teresa Zhang Yoonho Lee Roshen Sanjay Nair Aviral Kumar Chelsea Finn

Department of Computer Science
Stanford University

Abstract

Reasoning over long document contexts remains a challenge for large language models (LLMs). Standard retrieval methods are inherently reactive, bottlenecking agents with complex, cross-document reasoning at inference time. Existing approaches that pre-compute corpus structure, such as knowledge graphs, often fall short due to their reliance on rigid heuristics such as identifying interdocument relationships via named entity recognition. We propose an *anticipatory* approach to memory: proactively reorganizing unstructured document corpora to efficiently serve diverse future queries. This pre-computation synthesizes and restructures information within documents in advance, significantly improving an answering agent’s downstream performance and inference efficiency. While agentic scaffolds (e.g., Claude Code) are heavily trained to navigate and extend filesystems while solving the task currently in front of them, they do not pre-construct one in anticipation of future queries that our approach aims to do. We introduce Librarian Models, a novel framework that restructures unstructured corpora into file-based memories arranged around the operations that LLM agents already perform well, such as reading summaries and following links. Librarian Models is trained to maximize downstream success rate via end-to-end reinforcement learning. Empirically, Librarian Models improves accuracy over the best performing baseline by 77% relative improvement on BrowseComp+ and additionally generalizes to out-of-distribution corpora such as BRIGHT.

1 Introduction

Large language models (LLMs) are increasingly tasked with reasoning over extensive corpora of documents, such as enterprise knowledge bases, complex software repositories, and user chat histories, that exceed today’s LLM context windows. In these domains, the information required to com-

plete a given task is rarely centralized; instead, it is dispersed and disconnected across many disparate sources. Thus, for each incoming request, agents repeatedly process and revisit the corpus from scratch. Can we instead enable systems to efficiently navigate these massive context spaces without incurring high, redundant computational costs each time a user poses a new query?

A standard approach to this challenge is retrieval-augmented generation (RAG) (Gao et al., 2023), in which a system responds to a query by searching raw documents, retrieving relevant passages, and synthesizing an answer. Recent work has strengthened this pipeline with multi-agent frameworks (Jiang et al., 2023; Asai et al., 2023; Yan et al., 2024; Edge et al., 2024; Singh et al., 2025). However, these methods remain largely reactive: each query triggers a new round of search, requiring the model to repeatedly rediscover useful cross-document relationships at inference time. Other approaches pre-structure the search space before inference, for example as a knowledge graph (Peng et al., 2025). These methods can improve retrieval, but they typically commit to a rigid set of abstractions involving entities, relationships, and communities (Keraghel et al., 2024). Such a structure is useful when future queries precisely align with the index, but it is less natural when queries require new task-specific intermediate artifacts. This raises a critical question: can we train an agent to *reorganize a corpus in advance* so future LLM agents can efficiently find, inspect, and verify the right evidence?

Motivated by approaches that shift computation across different aspects of the model training and deployment process (Lin et al., 2025; Wu et al., 2026; Suzgun et al., 2025; Zhang et al., 2025a), we study *anticipatory memory construction*: the problem of augmenting and reorganizing a corpus *before* future queries arrive. The goal is to shift part of the search, synthesis, and organization costs

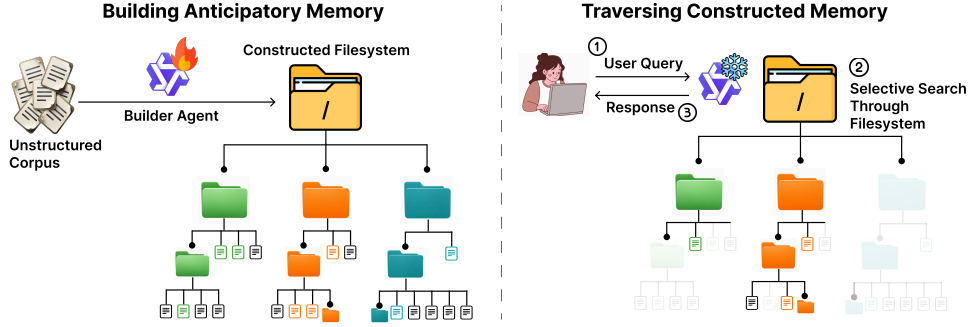


Figure 1: **Overview of Librarian Models.** (Left) We train a builder agent to construct an augmented filesystem consisting of synthetic files and directories that *anticipates* future queries for a downstream answering agent. (Right) This filesystem is built once and repeatedly used to answer different queries.

from query time to a precomputation phase, thereby amortizing these costs across many downstream queries. To instantiate this setting, we introduce Librarian Models, a framework built on a key asymmetry in current agentic systems: LLM agents are effective at navigating existing hierarchical filesystems, but are rarely trained to *proactively construct such workspaces for future use*. Librarian Models addresses this gap by tasking a **builder agent** with transforming source documents into an augmented, multi-level filesystem with provenance to the original corpus. An **answering agent** then navigates this filesystem to answer an individual user query. We formalize the builder’s operations as a Markov Decision Process (MDP) and directly optimize the memory construction policy for downstream answering performance by posing it as a reinforcement learning (RL) problem on this MDP.

We demonstrate the efficacy of Librarian Models on the BrowseComp-Plus domain, where compared to the strongest baseline for each answering agent, Librarian Models improves accuracy by an average of 33.0 percentage points, corresponding to a 76.7% relative improvement. Furthermore, for BRIGHT, a domain which our builder agent is untrained on, gains persist over baselines.

In summary, our key contributions are:

- We formulate memory anticipation as a desirable model behavior: given long corpora, agents should materialize useful context *before* receiving user queries.
- We propose Librarian Models, a filesystem-based framework, where a builder agent synthesizes and organizes memory artifacts.
- We formulate filesystem-based memory construction as an MDP and optimize the builder agent via reinforcement learning to improve

downstream query-answering performance.

- We evaluate Librarian Models on the BrowseComp-Plus and Bright benchmarks, showing 77% relative improvement over the strongest baseline on average.

2 Librarian Models: Agent-Augmented Filesystems as Anticipatory Memories

We introduce Librarian Models, a framework for training an agent to construct *anticipatory memories*: corpus representations built before future queries arrive. Document-grounded agents waste compute when each new query forces them to re-discover the same information across the raw corpus. Anticipatory memory shifts part of this work into a reusable construction phase. We instantiate this memory as an augmented filesystem, which exposes a long-context structure through operations LLM agents already use well: scanning filenames, opening concise artifacts, traversing directories, and checking claims against source files. The builder converts a flat collection of documents into a multi-level workspace whose synthetic files reduce search friction while preserving fallback paths to the original corpus. We refer to this builder as a **Librarian Model**: an agent trained to organize a corpus for future use.

We first define the document corpus, filesystem, and augmentation operations that transform a flat collection of raw documents into an answer-facing memory structure. We then articulate desiderata of an effective anticipatory memory, which includes diverse synthesis of augmentations, efficient navigation, fallback to raw evidence, and reuse across queries. Finally, we cast memory construction as a finite-horizon Markov decision process in which the librarian model edits the filesystem to maximize the answering agent’s expected performance,

subject to a construction-cost penalty.

2.1 Problem Setup.

We study long-context reasoning over a corpus of raw documents, $\mathcal{D} = \{d_1, d_2, \dots, d_n\}$, where the system must answer queries $q \sim \mathcal{Q}$. This setting matches retrieval-augmented generation (RAG) systems, which typically answer each query by searching a retrieval index over $\mathcal{D}$ and then synthesizing a response.

We represent the agent’s external memory as a *filesystem* $\mathcal{F}$: a set of files and directories together with metadata specifying each node’s type and parent. The initial filesystem $\mathcal{F}_0$ contains the raw documents in $\mathcal{D}$ as files under the root. A **filesystem augmentation** $\mathcal{A}$ is an edit operation that modifies a filesystem, for example by creating, rewriting, deleting, or moving files and folders. At iteration i , an augmentation $\mathcal{A}_i$ transforms the previous filesystem $\mathcal{F}_{i-1}$ into $\mathcal{F}_i$. Applying a sequence of augmentations for horizon H produces a final memory $\mathcal{F}_H$.

System performance is measured by a scoring function $\text{Acc}(\tilde{y}, y^*)$, which evaluates the correctness of the generated response $\tilde{y}$ against a ground-truth target y^* . Memory construction may also incur a cost, denoted $\text{Cost}(\mathcal{A}_i)$, such as the number of tokens generated or modified by $\mathcal{A}_i$.

Overview of Problem Setup

Anticipatory memory construction is the problem of transforming a large corpus into a structured filesystem memory before downstream queries are observed.

2.2 Desiderata for Anticipatory Memories.

A useful anticipatory memory should do more than “remember” a corpus. It should reshape the corpus into an *answer-facing workspace* that helps future agents quickly find likely evidence, while preserving a path back to raw source documents. As a running example, consider an enterprise corpus containing design docs, issue threads, launch notes, and customer reports for the same product. One future query may ask for a low-level implementation detail; another may ask how a user-visible behavior changed across releases; a third may ask for the exact source that supports a disputed claim. We use these examples to motivate three key desiderata.

Coverage with Efficient Navigation. The memory should cover many plausible queries while re-

ducing the need to search the raw corpus. In the enterprise example, the builder might create a release timeline, a component-level design index, and customer-facing behavior summaries, each linking the same underlying documents from a different angle. These artifacts help only if the answering agent can reach them through short, legible paths, such as directories named by abstraction level. Thus, the builder should synthesize cross-document artifacts that cover likely regions of the query distribution $\mathcal{Q}$ and place them where an agent can discover them with shallow traversal.

Faithful Fallback. No finite memory can anticipate *every* query, and synthesized artifacts can omit details that are critical in hindsight. The filesystem must therefore preserve provenance links and raw source paths so the answering agent can verify claims or bypass the synthesized layer when needed. In the enterprise example, a rare query about a single customer may not appear in the high-level summary, but the agent should still be able to read any existing issue thread. As an extreme example, such a fallback is important for needle-in-a-haystack queries where the answer depends on a localized, very specific fact that is unlikely to survive compression (Wang et al., 2024).

Amortized Reuse. The builder’s construction cost should pay off across many downstream queries. For example, a compact release timeline written by the builder agent may help answer many questions about regressions, roadmap changes, customer messaging, and dependency updates. In contrast, a one-off answer draft may surface similar information but help only a single request. This creates a trade-off between the construction cost $\sum_{i=1}^H \text{Cost}(\mathcal{A}_i)$ and the downstream utility of the final memory.

Overview of Memory Desiderata

An effective anticipatory memory should make likely queries easy to answer, preserve fallback paths to raw documents, and pay off across many downstream queries.

2.3 Memory MDP Formalization.

In Librarian Models, anticipatory memory construction proceeds in two phases: an upfront synthesis phase and a downstream answering phase. The synthesis phase is analogous to the offline construction of a retrieval index or knowledge graph, but instead produces a flexible filesystem memory. To instantiate this idea, we define a **builder**

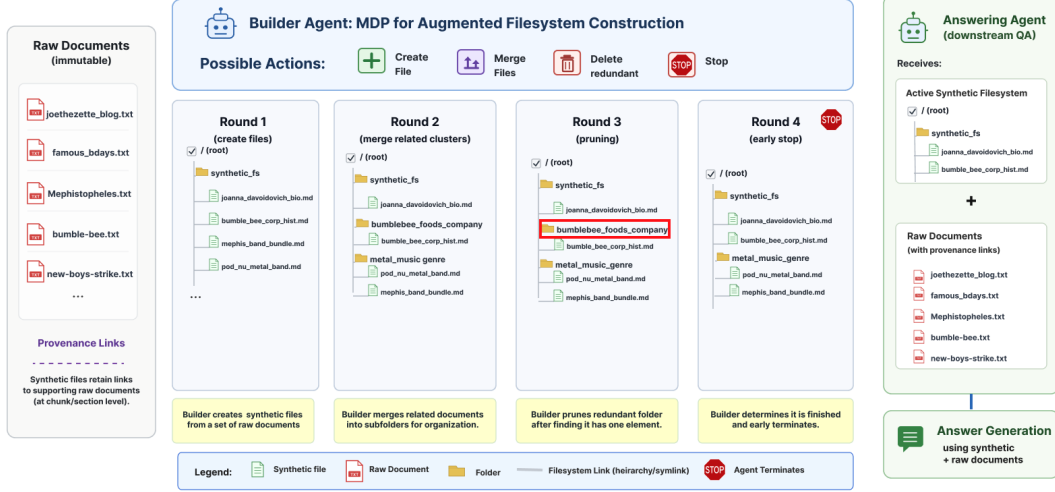


Figure 2: **MDP Definition of Librarian Models.** Starting from a raw list of documents, a builder agent can create a filesystem with 4 defined actions: Create, Merge, Delete, and Stop. This filesystem is fed into the context of an answering agent which navigates the filesystem to output a response, which achieves a reward on a given task. Here, we depict an augmented filesystem from the BrowseComp-Plus dataset.

agent $\pi_{\text{build}}(\mathcal{A} \mid \mathcal{F})$, which proposes filesystem augmentations given the current filesystem, and an **answering agent** $\pi_{\text{ans}}(\tilde{y} \mid q, \mathcal{F})$, which generates an answer $\tilde{y}$ for a new query q using the constructed filesystem.

We cast the builder agent’s sequence of filesystem operations as a finite-horizon Markov Decision Process (MDP), allowing memory construction to be optimized as a policy-learning problem.

State, Action, and Transition Dynamics. At each timestep t , the state is the current filesystem $\mathcal{F}_t$, consisting of the original raw documents $\mathcal{D}$ together with the synthetic files, folders, and provenance links created so far. The builder agent selects an augmentation $\mathcal{A}_t \sim \pi_{\text{build}}(\cdot \mid \mathcal{F}_t)$, which is applied to the filesystem to produce $\mathcal{F}_{t+1}$. We constrain the builder to the following discrete action types that can each be applied to different files:

- **CREATEFILE:** Creates a new synthetic file by consolidating information from a subset of raw documents and/or existing synthetic files. The file links to sources that construct it.
- **MERGEFILES:** Aggregates a set of related synthetic files into a higher-level summary or directory-like artifact. This action introduces hierarchy, linking to child files.
- **DELETE:** Removes redundant/unhelpful synthetic files from the active answer-facing filesystem. This operation never deletes raw documents in $\mathcal{D}$, preserving fallback access.

- **STOP:** Terminates memory construction, returning the current filesystem $\mathcal{F}_t$ as the final memory artifact.

The action types are intentionally constrained to simplify optimization. In principle, this interface can be generalized to a broader set of atomic filesystem commands (e.g., touch, mkdir, mv, rm, etc). We describe the instantiation of these actions in the filesystem in Appendix A.6.

Reward Function. The reward should incentivize the builder to construct a memory artifact that improves downstream question answering across the query distribution $\mathcal{Q}$. A natural sparse reward formulation scores the final filesystem $\mathcal{F}_H$ by the expected success of the answering agent using it, while penalizing the total construction cost incurred by the sequence of augmentations:

$$R(\mathcal{F}_H) := S(\mathcal{F}_H) - \alpha \sum_{t=1}^H \text{Cost}(\mathcal{A}_t), \quad (1)$$

where

$$S(\mathcal{F}) := \mathbb{E}_{q \sim \mathcal{Q}} [\mathbb{E}_{\tilde{y} \sim \pi_{\text{ans}}(\cdot \mid q, \mathcal{F})} [\text{Acc}(\tilde{y}, y^*(q))]].$$

Here, $S(\mathcal{F})$ measures the expected success rate of the answering agent when using filesystem $\mathcal{F}$, $y^*(q)$ denotes the ground-truth answer for query q , and α controls the tradeoff between downstream accuracy and construction cost. In practice, we will estimate the expectation over queries using a finite validation set of sampled queries.

While this sparse objective directly captures the desired end behavior, it provides limited credit assignment during construction. The builder only observes whether the final filesystem is useful after a potentially long sequence of edits, making it difficult to determine which intermediate augmentations improved downstream answering performance. To provide a denser learning signal, we define the reward of an individual augmentation as its marginal contribution to answering success. After applying augmentation $\mathcal{A}_t$ to transform $\mathcal{F}_{t-1}$ into $\mathcal{F}_t$, we assign

$$r_t^{\text{dense}} := [S(\mathcal{F}_t) - S(\mathcal{F}_{t-1})] - \alpha \text{Cost}(\mathcal{A}_t). \quad (2)$$

This reward directly measures whether each filesystem augmentation improves the answering agent’s expected success rate on the target query distribution. By telescoping, maximizing the cumulative dense reward is equivalent to maximizing the sparse terminal objective up to the constant baseline $S(\mathcal{F}_0)$. We use a discount factor of $\gamma = 1$.

Overview of MDP Formulation

We formulate memory construction as a dense-reward MDP in which the builder agent is incentivized to maximize downstream utility at low construction cost.

2.4 Memory Augmentations via RL.

The MDP formulation above enables direct optimization of the memory-construction process with reinforcement learning. In each rollout, the builder iteratively edits the filesystem for up to H steps, producing a trajectory

$$\tau = (\mathcal{F}_0, \mathcal{A}_1, \mathcal{F}_1, \dots, \mathcal{A}_T, \mathcal{F}_T), \quad (3)$$

where $T \leq H$ is determined either by the horizon or the STOP action.

The resulting filesystem is then evaluated by the answering agent on a minibatch of queries sampled from $\mathcal{Q}$. These queries are used only for reward estimation and are **not** provided to the builder during construction. The builder receives either the terminal reward in Eq. 1 or the dense marginal reward in Eq. 2. This training signal encourages the builder to discover filesystem edits that are not merely locally coherent but also useful for improving downstream question answering after accounting for construction cost. We optimize the builder policy with GRPO (Shao et al., 2024), a policy-gradient method used for RL-based fine-tuning of language models.

Algorithm 1 RL Training of Librarian Models

Require: Documents $\mathcal{D}$, query distribution $\mathcal{Q}$, builder π_{build} , answerer π_{ans} , horizon H

Ensure: Optimized builder policy π_{build}

```

1: for  $k = 1, \dots, K$  do
2:   Sample query batch  $\mathcal{B}_k \sim \mathcal{Q}$ 
3:   Initialize  $\mathcal{F}_0 \leftarrow (\mathcal{D}, \emptyset)$ ,  $\tau \leftarrow \emptyset$ 
4:   for  $t = 1, \dots, H$  do
5:     Sample edit  $\mathcal{A}_t \sim \pi_{\text{build}}(\cdot \mid \mathcal{F}_{t-1})$ 
6:     if  $\mathcal{A}_t = \text{STOP}$  then
7:       break
8:     Apply edit:  $\mathcal{F}_t \leftarrow \mathcal{A}_t(\mathcal{F}_{t-1})$ 
9:     Compute dense reward  $\hat{r}_t$ 
10:    Append  $(\mathcal{F}_{t-1}, \mathcal{A}_t, \hat{r}_t, \mathcal{F}_t)$  to  $\tau$ 
11:   Update  $\pi_{\text{build}}$  with GRPO on rollout  $\tau$ 
12: return  $\pi_{\text{build}}$ 

```

By default, we treat the answering agent π_{ans} as fixed during builder optimization. This isolates the learning problem: the builder learns to construct memories useful to the answering agent, which may be a language model or another long-context reasoning system such as RLM (Zhang et al., 2025a). We can also consider an alternative joint-training variant in which the answering agent is adapted using the same downstream correctness signal. In this setting, optimization can alternate between updating the builder and updating the answerer via cooperative multi-agent RL framework, following methods such as RLAD (Qu et al., 2025a). The full optimization process for Librarian Models is outlined in Algorithm 1.

Overview of RL Optimization

We optimize the builder as an LLM policy over filesystem edits. Each rollout constructs a memory, evaluates its downstream utility with a fixed answering agent, and updates the builder with GRPO using either terminal or dense rewards.

3 Experiments

Having formulated anticipatory memory construction as an MDP in Section 2, we evaluate whether Librarian Models improves the reasoning performance and inference efficiency of downstream answering agents on long-context document reasoning tasks. We compare Librarian Models with prior reactive retrieval and pre-computation methods to address four questions: (1) Do Librarian Mod-

els improve in-distribution long-context reasoning performance? (2) What amortization benefits do Librarian Models provide when serving multiple queries over a shared document set? (3) Does the builder agent’s anticipatory construction capability generalize zero-shot to out-of-distribution retrieval settings? (4) Qualitatively, how do the builder and answering agents behave during the construction and answering phases, and do these behaviors reflect the intended anticipatory capability? We examine each question in turn below.

Task Definition. The main domain we consider for long-document reasoning is BrowseComp-Plus (Chen et al., 2025), a curated corpus of deep-research web tasks. We split this dataset into two sets for training and evaluation: 680 tasks for training and 50 held-out tasks for all evaluations (sampled at random). For training of π_{build} , each task consists of an independent set of raw documents and a downstream query that needs to be answered about the set of documents. We additionally evaluate the performance of Librarian Models on the BRIGHT Benchmark (Su et al., 2025), which comprises a set of tasks from diverse domains (Stack-Exchange, LeetCode, and math competitions), to assess the builder agent’s efficacy on unseen distributions.

Comparisons and baselines. We evaluate our approach against several representative baselines. First, we compare with standard reactive retrieval methods: an *Embedding* with optional *Reranker* baseline (Zhang et al., 2025c) and *Late Interaction Embedding* (Chaffin and Sourty, 2025), which represent conventional retrieve-and-read RAG systems that process each query from scratch. Second, we compare with *Graph RAG* (Peng et al., 2025), a knowledge graph baseline that uses precomputation. Third, we include *Agentic RAG* (Singh et al., 2025), which represents multi-agent approaches that dynamically interleave retrieval and reasoning at query time. Next, we compare autocompaction method such as those that periodically summarize the computation performed, such as RC (Wu et al., 2026). These baselines help quantify the trade-offs of deferring structural reasoning to inference time to a summarization agent.

We also include an ablation of our own framework: a *builder agent without RL fine-tuning*. This isolates the contribution of our MDP-based policy optimization. For all methods, the downstream answering agents, Qwen3.5-4B and Gemini 3.1 Flash, are kept frozen so that performance differ-

ences reflect only the quality of the resulting memory artifacts. More details about the baselines and hyperparameters can be found in Appendix A.4.

(1) In Distribution Performance of Librarian Models on BrowseComp-Plus. We evaluate the performance of Librarian Models on the BrowseComp-Plus domain in Table 1. Here we find that Librarian Models with RL fine-tuning significantly outperforms all reactive and pre-computation baselines. When using the frozen Qwen3.5-4B answering agent, Librarian Models achieves an accuracy of 0.75, representing a substantial improvement over the Embedding (0.42) and Reranker (0.44) baselines. A similar trend is observed with Gemini 3.1 Flash Lite, where Librarian Models reaches 0.77 compared to the baseline scores of 0.30 and 0.42. Crucially, the ablation without RL fine-tuning (No FT) additionally underperforms below Librarian Models, highlights that policy optimization with RL is essential for teaching the builder agent to structure anticipatory memories effectively for downstream consumption, a task the base model was not trained for.

(2) Amortization benefit of Librarian Models. A primary motivation for decoupling memory construction from query execution is the ability to amortize costs when processing multiple queries against a single document set. As shown in Table 2, while Librarian Models incurs an upfront memory construction cost of 179K tokens, falling between our GraphRAG and AutoCompaction baselines, it achieves a significantly higher overall accuracy of 0.77. To demonstrate how this initial cost can be distributed over the same document set, we conducted an evaluation using 20 synthetic queries per document set on BrowseComp-Plus generated by Gemini 3.1 Flash Lite, fewshot prompted with sample browsecomp questions. Running these queries on the same evaluation filesystem yielded an accuracy of 0.64, while reducing the *amortized cost per query* to 9.3K tokens (full results in Table 6), illustrating the efficiency and reusability of Librarian Models’s builder agent.

(3) Transfer of Builder Agent to novel long-context document reasoning setting. To assess whether the builder agent learns a generalized anticipatory capability rather than simply overfitting to the DeepResearch training distribution, we evaluate Librarian Models zero-shot on the BRIGHT benchmark. As detailed in Table 3, the RL-finetuned builder agent successfully transfers to highly out-of-distribution domains, including biomedical texts,

Answering Agent	Reactive Retrieval / RAG						Caching	Builder-Based Memory	
	Embed.	Hybrid	LateInter.	Rerank.	Graph	Agentic	AutoComp.	No FT	RL (Ours)
Qwen3.5-4B, Frozen	0.42	0.24	0.38	0.44	0.26	0.44	0.02	0.25	0.75
Gemini 3.1 Flash Lite	0.30	0.18	0.30	0.42	0.36	0.42	0.04	0.28	0.77

Table 1: **Results on BrowseComp-Plus.** We compare Librarian Models against RAG, Caching and Builder-Based Memory baselines using two answering agent models (Qwen 3.5 4B and Gemini 3.1 Flash Lite). We find that Librarian Models significantly outperforms all methods for both answering agents. For computing accuracy, we generate 4 responses per query.

Memory / Retrieval	Acc.	Construction Cost (tokens)
GraphRAG	0.36	11,923
AutoCompaction	0.04	501,637
Librarian Models (ours)	0.77	178,926

Table 2: **Construction Cost Analysis on BrowseComp-Plus.** We compare the construction cost of methods that have a construction phase such as GraphRag and AutoCompaction. All methods use the same frozen answering agent. Construction cost denotes the upfront construction cost in tokens.

economics, and theorem proving. Librarian Models achieves an average zero-shot accuracy of 0.61 across the 12 BRIGHT datasets, outperforming the No Fine-Tuning baseline by a significant margin of 31%. These results confirm that framing memory construction as an MDP enables the builder to learn fundamental, transferable heuristics for organizing information, which remain robust even when encountering novel, unseen documents.

(4) Analysis of Librarian Models and capability of anticipation. Finally, we wish to analyze the form of filesystems that Librarian Models constructs. We start off by considering what distribution of files that the builder agent constructs for a given prompt. As seen in Figure 2, we see that the builder adds a mixture of breadth of synthetic files and depth via organization through directories to the constructed filesystem in its augmentations. The breadth explicitly enables the capability of anticipation: capturing a wider distribution of queries where as the depth allows the model to handle fine-grained queries. We explicitly measure this in Table 4, studying the behavior of builder and answering agents respectively. Here, we list the number of synthetic files/tokens generated and visited by both agents as well as the depth and the breadth of the filesystem. Furthermore, in Figure 4, where in the BrowseComp+ domain, the agent has a fallback when the query is too fine-grained and looks at the raw files from the unstructured document corpus to make a decision. Despite this reliance on the source document, the agent still navigates the filesystem efficiently without needing to enumerate all unstructured documents in the corpus. Finally, we compare the behavior of Li-

brarian Models and the base policy (No FT) on the BrowseComp+ domain in Figure 5. Here, we find that the builder agent generates a more expansive corpus that captures a wider distribution of queries.

Overview of Experimental Evaluation

Our evaluation demonstrates that Librarian Models significantly improves long-context document reasoning and amortizes per-query costs on both in-distribution tasks like BrowseComp-Plus and out-of-distribution domains such as BRIGHT.

4 Related Work

Retrieval-Augmented Generation. RAG grounds language model outputs in external evidence, mitigating the limitations of purely parametric memory (Lewis et al., 2020; Gao et al., 2023; Li et al., 2025; Oche et al., 2025). Recent innovations have evolved this pipeline into agentic systems capable of multi-step reasoning (Xu et al., 2024; Zhang et al., 2025b; Singh et al., 2025). For instance, models can now predict upcoming content to trigger searches (Jiang et al., 2023), use reflection tokens for self-critique (Asai et al., 2023; Yan et al., 2024), or execute planned searches to navigate multi-hop questions (Li et al., 2026). However, these approaches share a reactive bottleneck: they initiate complex, iterative retrieval only *after* receiving a user query. In contrast, Librarian Models shifts this computational burden by proactively synthesizing a structured, query-agnostic filesystem *prior* to inference, amortizing the cost of corpus-level reasoning.

Structured Indexes and Corpus-Level Organization. To navigate vast text collections, prior systems often pre-compute structured representations. Knowledge graph approaches extract entity networks and community summaries for global document understanding (Edge et al., 2024; Li et al., 2024; Guo et al., 2025), while others use biologically inspired retrieval (Jiménez Gutiérrez et al., 2024). Although valuable, these preprocessing methods rely heavily on rigid, rule-based ex-

Memory / Retrieval	StackExchange							Coding		Theorems			Avg. Acc.	Cost / Query
	Bio.	Earth Sci.	Econ.	Psych.	Robot.	StackOv.	Sust. Liv.	LeetCode	Pony	AoPS	ThmQA-Q	ThmQA-T		
Qwen3.5-4B Builder	0.338	0.263	0.163	0.325	0.237	0.525	0.125	0.287	0.487	0.150	0.350	0.388	0.303	919.96
Librarian Models (ours)	0.650	0.738	0.725	0.725	0.550	0.812	0.438	0.562	0.725	0.225	0.637	0.562	0.613	8109.34

Table 3: **Zero-Shot Transfer to BRIGHT.** The builder agent, trained only on DeepResearch tasks (BrowseComp) is evaluated on BRIGHT without task-specific training, testing whether Librarian Models learns a transferable capability of anticipation across heterogeneous query and document sources. Librarian Models outperforms the base builder model on all tasks within BRIGHT (StackExchange, Coding, and Theorems) achieving an average accuracy of 61.3%.

traction pipelines or predefined schemas. In contrast, Librarian Models make anticipatory memory construction fully agentic and optimizable. Instead of static graph heuristics, our builder agent uses RL to generate synthetic filesystem augmentations—dynamically synthesizing, merging, or pruning documents to strictly optimize the corpus for downstream utility.

Long-Term Memory for Language Agents. Prior work explores endowing language models with persistent, stateful memory to support continuous operation. For instance, recent works propose to cache conversational histories (Wang et al., 2023; Zhong et al., 2023; Uddin et al., 2026), manage virtual context windows via memory tiers (Packer et al., 2023), and unify short- and long-term memory directly within the agent’s policy (Yu et al., 2026; Xu et al., 2025). However, these approaches primarily focus on an agent retaining its own historical context. In contrast, Librarian Models studies anticipatory memory construction over external corpora. Rather than simply logging past interactions, a builder agent proactively synthesizes, organizes, and prunes an answer-facing filesystem before queries arrive. This generates a reusable memory artifact for a separate answering agent, explicitly optimizing the trade-off between downstream utility and construction cost.

Reasoning, Acting, and Agentic Learning. Integrating internal reasoning with external tools has vastly expanded agent capabilities (Wei et al., 2022; Yao et al., 2023, 2022), enabling systems like Toolformer (Schick et al., 2023) and Reflexion (Shinn et al., 2023) to invoke APIs and self-correct via feedback. While Librarian Models also utilizes tools (cluster, merge, delete), it optimizes a fundamentally different MDP: rather than acting to directly solve a user prompt, our builder focuses strictly on constructing a filesystem memory artifact for a future answering agent to traverse.

Compute Reallocation. Our approach aligns with emerging literature on reallocating computation across time to improve test-time scaling (Lin et al., 2025; Wu et al., 2026). Other approaches for-

malize test-time compute optimization as meta-reinforcement learning optimization to adaptively allocate tokens (Qu et al., 2025b) or utilize self-curated test-time curricula to reinforce successful strategies (Hübner et al., 2025). While these methods share the goal of using compute to prevent reactivity, Librarian Models uniquely formulates this shift as a synthetic filesystem construction problem. We model the memory building of a filesystem as an MDP, which is optimized using RL. Furthermore, Librarian Models makes the compute reallocation problem an explicit target of policy optimization rather than a byproduct of inference.

5 Discussion and Future Work

We study the capability of *anticipatory memory construction*, where agents can proactively reorganizing their context into a memory artifact optimized for future queries the agent encounters. To instantiate this capability, we rely on an asymmetry for agents today, where agents are trained to navigate a filesystem for a given task, yet do not pre-construct filesystems for future queries that they might encounter. Our novel framework, Librarian Models, introduces builder agent to create reusable memory artifact in the form a synthetic filesystem that an answering agent can navigate. Librarian Models yields consistent improvements across several long-context document reasoning benchmarks such as BrowseComp+ and BRIGHT, outperforming existing methods for retrieval on both in-domain and out-of-distribution evals.

Moving forward, several promising avenues for future work exist. Our primary experiments treated the answering agent as fixed to isolate the learning problem of the builder. Future research could explore a variant where both agents are co-adapted using the same downstream correctness signal. Finally, although the current builder is constrained to discrete actions to simplify optimization, this interface could be generalized to a broader set of atomic filesystem commands (e.g bash), enabling even more sophisticated memory structures.

6 Limitations

A limitation of this work is that our current evaluation is restricted to question-answering over static corpora. Maintaining and updating anticipatory memories in dynamic environments where documents are frequently added, modified, or removed remains an open problem. Furthermore, the builder agent must currently operate over a large context of files to construct the synthetic filesystem, which may limit the framework’s feasibility and scalability as the corpus size blows up. To ameliorate this, approaches like Librarian Models can be paired with clustering or retrieval methods to seamlessly handle massive enterprise domains containing billions of files. Additionally, no finite set of synthesized memory artifacts can anticipate every possible future query. The synthesis process inherently involves compression, which can discard fine-grained details necessary for highly specific "needle-in-a-haystack" tasks. While we mitigate this by preserving provenance links and raw source paths for fallback access, navigating the exact threshold of when an agent should forgoe the synthetic filesystem for the set of raw documents remains a challenge.

7 Ethical considerations

Anticipatory memory construction via Librarian Models may introduce some ethical risks. Firstly because the builder agent’s synthesis operations (such as creating and merging files) rely on compression, information is necessarily discarded. There is a risk that the builder may synthesize hallucinated facts or inadvertently misrepresent the original text during this phase. If this occurs, downstream answering agents navigating the flawed filesystem may confidently produce incorrect answers. Although our framework mitigates this by preserving the provenance of the original documents, giving the answering agent the option to fall back to raw evidence, there is a risk that the answering agent might over-rely on the synthetic artifacts rather than rigorously verifying claims against the set of source documents.

CS224R Contribution Statement

For the CS224R final project, I received permission from course staff and from the project collaborators to use Librarian Models, a research project that I have worked on in the IRIS Lab over the past two quarters. For transparency, the main technical paper included in this report is also the research manuscript prepared from that project, and I received permission to use it as the main paper for my CS224R submission. Below, I clarify my personal contributions to the project, particularly for this quarter.

Personal Contributions. During winter quarter, my contributions included helping formulate the project direction, building the required datasets, and running pilot experiments on earlier iterations of the project idea. During spring quarter, my main contributions were designing, implementing, and running the reinforcement-learning experiments, including the training runs, evaluation runs, and ablations with frozen base models. In addition, my work this quarter included contributing to the final design of the filesystem-builder MDP, interpreting and analyzing the experimental results, and helping prepare the final paper.

External Collaborators. Anikait Singh, who is also a CS224R TA, and Yoonho Lee were my main mentors for this project, and they are the collaborators I worked with most directly over the past two quarters. In addition, all contributors to the broader research project are listed as authors on the main technical paper.

References

- Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2023. [Self-RAG: Learning to retrieve, generate, and critique through self-reflection](#). *arXiv preprint arXiv:2310.11511*.
- Antoine Chaffin and Raphaël Sourty. 2025. [Pylate: Flexible training and retrieval for late interaction models](#). *Preprint*, arXiv:2508.03555.
- Zijian Chen, Xueguang Ma, Shengyao Zhuang, Ping Nie, Kai Zou, Andrew Liu, Joshua Green, Kshama Patel, Ruoxi Meng, Mingyi Su, and 1 others. 2025. Browsecomp-plus: A more fair and transparent evaluation benchmark of deep-research agent. *arXiv preprint arXiv:2508.06600*.
- Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. [From local to global: A graph RAG approach to query-focused summarization](#). *arXiv preprint arXiv:2404.16130*.
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2023. [Retrieval-augmented generation for large language models: A survey](#). *arXiv preprint arXiv:2312.10997*.
- Kai Guo, Harry Shomer, Shenglai Zeng, Haoyu Han, Yu Wang, and Jiliang Tang. 2025. [Empowering GraphRAG with Knowledge Filtering and Integration](#). *Preprint*, arXiv:2503.13804.
- Jonas Hübner, Leander Diaz-Bone, Ido Hakimi, Andreas Krause, and Moritz Hardt. 2025. [Learning on the Job: Test-Time Curricula for Targeted Reinforcement Learning](#). *Preprint*, arXiv:2510.04786.
- Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. [Active retrieval augmented generation](#). *arXiv preprint arXiv:2305.06983*.
- Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. [HippoRAG: Neurobiologically inspired long-term memory for large language models](#). *arXiv preprint arXiv:2405.14831*.
- Imed Keraghel, Stanislas Morbieu, and Mohamed Nadif. 2024. [Recent advances in named entity recognition: A comprehensive survey and comparative study](#). *Preprint*, arXiv:2401.10825.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. [Retrieval-augmented generation for knowledge-intensive nlp tasks](#). *arXiv preprint arXiv:2005.11401*.
- Mufei Li, Siqi Miao, and Pan Li. 2024. [Simple Is Effective: The Roles of Graphs and Large Language Models in Knowledge-Graph-Based Retrieval-Augmented Generation](#). *Preprint*, arXiv:2410.20724.
- Siran Li, Linus Stenzel, Carsten Eickhoff, and Seyed Ali Bahrainian. 2025. [Enhancing Retrieval-Augmented Generation: A Study of Best Practices](#). *Preprint*, arXiv:2501.07391.
- Xingyu Li, Rongguang Wang, Yuying Wang, Mengqing Guo, Chenyang Li, Tao Sheng, Sujith Ravi, and Dan Roth. 2026. [PAR²-RAG: Planned Active Retrieval and Reasoning for Multi-Hop Question Answering](#). *Preprint*, arXiv:2603.29085.
- Kevin Lin, Charlie Snell, Yu Wang, Charles Packer, Sarah Wooders, Ion Stoica, and Joseph E. Gonzalez. 2025. [Sleep-time compute: Beyond inference scaling at test-time](#). *arXiv preprint arXiv:2504.13171*.
- Agada Joseph Oche, Ademola Glory Folashade, Tirthankar Ghosal, and Arpan Biswas. 2025. [A Systematic Review of Key Retrieval-Augmented Generation \(RAG\) Systems: Progress, Gaps, and Future Directions](#). *Preprint*, arXiv:2507.18910.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. [MemGPT: Towards LLMs as operating systems](#). *arXiv preprint arXiv:2310.08560*.
- Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2025. Graph retrieval-augmented generation: A survey. *ACM Transactions on Information Systems*, 44(2):1–52.
- Yuxiao Qu, Anikait Singh, Yoonho Lee, Amrith Setlur, Ruslan Salakhutdinov, Chelsea Finn, and Aviral Kumar. 2025a. [Rlad: Training llms to discover abstractions for solving reasoning problems](#). *Preprint*, arXiv:2510.02263.
- Yuxiao Qu, Matthew Y. R. Yang, Amrith Setlur, Lewis Tunstall, Edward Emanuel Beeching, Ruslan Salakhutdinov, and Aviral Kumar. 2025b. [Optimizing Test-Time Compute via Meta Reinforcement Fine-Tuning](#). *Preprint*, arXiv:2503.07572.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. [Toolformer: Language models can teach themselves to use tools](#). *arXiv preprint arXiv:2302.04761*.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, and 1 others. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. [Reflexion: Language agents with](#)

- verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*.
- Aditi Singh, Abul Ehtesham, Saket Kumar, Tala Talaie Khoei, and Athanasios V. Vasilakos. 2025. [Agentic retrieval-augmented generation: A survey on agentic RAG](#). *arXiv preprint arXiv:2501.09136*.
- Hongjin Su, Howard Yen, Mengzhou Xia, Weijia Shi, Niklas Muennighoff, Han Yu Wang, Haisu Liu, Quan Shi, Zachary S. Siegel, Michael Tang, Ruoxi Sun, Jinsung Yoon, Serkan O. Arik, Danqi Chen, and Tao Yu. 2025. [Bright: A realistic and challenging benchmark for reasoning-intensive retrieval](#). *Preprint*, arXiv:2407.12883.
- Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. 2025. [Dynamic cheat-sheet: Test-time learning with adaptive memory](#). *Preprint*, arXiv:2504.07952.
- Md Nayem Uddin, Kumar Shubham, Eduardo Blanco, Chitta Baral, and Gengyu Wang. 2026. [From Recall to Forgetting: Benchmarking Long-Term Memory for Personalized Agents](#). *Preprint*, arXiv:2604.20006.
- Weiyun Wang, Shuibo Zhang, Yiming Ren, Yuchen Duan, Tiantong Li, Shuo Liu, Mengkang Hu, Zhe Chen, Kaipeng Zhang, Lewei Lu, Xizhou Zhu, Ping Luo, Yu Qiao, Jifeng Dai, Wenqi Shao, and Wenhai Wang. 2024. [Needle in a multimodal haystack](#). *Preprint*, arXiv:2406.07230.
- Weizhi Wang, Li Dong, Hao Cheng, Xiaodong Liu, Xifeng Yan, Jianfeng Gao, and Furu Wei. 2023. [Augmenting language models with long-term memory](#). *arXiv preprint arXiv:2306.07174*.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. [Chain-of-thought prompting elicits reasoning in large language models](#). *arXiv preprint arXiv:2201.11903*.
- Ian Wu, Yuxiao Qu, Amrith Setlur, and Aviral Kumar. 2026. [Reasoning cache: Continual improvement over long horizons via short-horizon RL](#). *arXiv preprint arXiv:2602.03773*.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. [A-MEM: Agentic Memory for LLM Agents](#). *Preprint*, arXiv:2502.12110.
- Zhipeng Xu, Zhenghao Liu, Yibin Liu, Chenyan Xiong, Yukun Yan, Shuo Wang, Shi Yu, Zhiyuan Liu, and Ge Yu. 2024. [ActiveRAG: Revealing the Treasures of Knowledge via Active Learning](#). *Preprint*, arXiv:2402.13547v1.
- Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. 2024. [Corrective retrieval augmented generation](#). *arXiv preprint arXiv:2401.15884*.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. [Tree of thoughts: Deliberate problem solving with large language models](#). *arXiv preprint arXiv:2305.10601*.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. [ReAct: Synergizing reasoning and acting in language models](#). *arXiv preprint arXiv:2210.03629*.
- Yi Yu, Liuyi Yao, Yuexiang Xie, Qingquan Tan, Jiaqi Feng, Yaliang Li, and Libing Wu. 2026. [Agentic Memory: Learning Unified Long-Term and Short-Term Memory Management for Large Language Model Agents](#). *Preprint*, arXiv:2601.01885.
- Alex L. Zhang, Tim Kraska, and Omar Khattab. 2025a. [Recursive language models](#). *arXiv preprint arXiv:2512.24601*.
- Chao Zhang, Yuhao Wang, Derong Xu, Haoxin Zhang, Yuanjie Lyu, Yuhao Chen, Shuochen Liu, Tong Xu, Xiangyu Zhao, Yan Gao, Yao Hu, and Enhong Chen. 2025b. [TeaRAG: A Token-Efficient Agentic Retrieval-Augmented Generation Framework](#). *Preprint*, arXiv:2511.05385.
- Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, and 1 others. 2025c. [Qwen3 embedding: Advancing text embedding and reranking through foundation models](#). *arXiv preprint arXiv:2506.05176*.
- Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2023. [MemoryBank: Enhancing large language models with long-term memory](#). *arXiv preprint arXiv:2305.10250*.

A Appendix

A.1 Additional Results

On BrowseComp+ and BRIGHT domains, we show the pass@k (coverage) performance for Librarian Models and the base model in Figure 3.

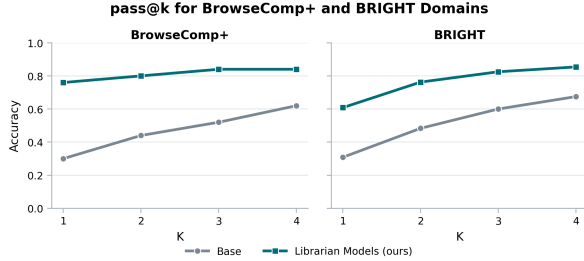


Figure 3: **Pass@K curve for the Builder Agent on BrowseComp-Plus and BRIGHT.** We plot the accuracy over parallel sample generations, showcasing an improvement of filesystem creation with additional test time inference.

Furthermore, we list statistics on the number of synthetic files and tokens created by the builder agent. We also include details about the breadth and depth of the constructed filesystems from the builder.

Domain	Model	Syn. Files	Syn. Tok.	Tot. Tok.	Act. Files	Clusters	Merges
Bio.	Base	2.7	1064	1189	5.4	4.9	0.5
	FT	30.4	9883	10091	54.0	45.8	8.2
Earth Sci.	Base	2.1	846	977	3.4	3.0	0.4
	FT	27.0	8769	8998	46.2	39.3	6.8
Econ.	Base	2.2	740	854	4.4	3.9	0.5
	FT	26.9	7438	7664	46.7	39.1	7.7
Psych.	Base	3.2	967	1118	4.3	3.8	0.5
	FT	29.4	8333	8578	46.5	39.5	7.0
Robot.	Base	2.5	805	956	4.0	3.3	0.7
	FT	23.9	6643	6889	37.6	31.9	5.7
StackOv.	Base	3.3	1075	1250	4.5	4.0	0.5
	FT	32.0	9246	9511	49.5	40.6	8.8
Sust. Liv.	Base	2.0	626	747	3.6	3.0	0.5
	FT	28.1	7554	7803	45.6	37.9	7.8
LeetCode	Base	4.8	1330	1498	6.2	5.4	0.8
	FT	39.6	9454	9702	59.8	50.6	9.2
Pony	Base	4.3	919	1070	4.6	4.3	0.3
	FT	35.4	7211	7474	50.1	43.9	6.3
AoPS	Base	3.9	879	1023	4.5	3.8	0.7
	FT	36.2	7309	7544	56.7	49.7	7.0
ThmQA-Q	Base	2.8	667	794	4.1	3.5	0.6
	FT	33.2	6468	6720	51.0	44.0	7.0
ThmQA-T	Base	4.4	1122	1300	6.0	4.9	1.1
	FT	42.3	9003	9269	64.1	53.3	10.7
Macro avg.	Base	3.2	920	1065	4.6	4.0	0.6
	FT	32.0	8109	8353	50.6	43.0	7.7

Table 4: **BRIGHT statistics by domain.** We report answerer retrieval behavior and final filesystem-structure statistics for each BRIGHT domain. Syn. Files, Syn. Tok., and Tot. Tok. measure files and tokens read by the answering agent, while Act. Files, Clusters, and Merges summarize the active synthetic filesystem produced by the builder.

Below in Table 6, we compare the performance of Librarian Models with a non-finetuned builder baseline on an amortized variant of BrowseComp+.

We also showcase a qualitative example of an agent recovering using Provenance Information,

Dataset	Model	Syn. Files	Syn. Tok.	Tot. Tok.	Act. Files	Clusters	Merges
BrowseComp+	Base	2.7	1069	1235	3.2	2.7	0.6
	FT	26.9	9040	9309	42.2	34.4	7.8

Table 5: **BrowseComp+ statistics.** We report the same statistics on the amortized BrowseComp+ evaluation. Syn. Files, Syn. Tok., and Tot. Tok. again measure answering-agent retrieval behavior, while Act. Files, Clusters, and Merges summarize the active synthetic filesystem produced by the builder.

BrowseComp+	Accuracy	Answer Tokens / Q	Files / Q
Qwen 3.5 4B Builder	0.24	1238	2.72
Ours	0.64	9346	26.90

Table 6: **Performance metrics for Amortized BrowseComp+ (Base vs. Fine-Tuned).** We achieve significantly higher accuracy than of 0.24 the base model on Amortized BrowseComp+, leading to a token budget of 9346 per query.

when the synthesized documents didn’t have the complete information to answer a question in Figure 4. Additionally, we compare a rollout of the builder trained with RL with the base model in Figure 5.

A.2 Potential Risks

Anticipatory memory construction via Librarian Models introduces several potential risks. Because compression necessarily discards information, no finite set of synthesized artifacts can cover every future query. If the builder synthesizes hallucinated facts or misrepresents the original text during the CREATEFILE or MERGEFILES actions, downstream answering agents may confidently produce incorrect answers. Although our framework preserves the provenance of the original documents giving the answering agent the option to fall back to raw evidence, there is a risk the answering agent might over-rely on the synthetic artifacts. Additionally, while anticipatory construction is designed to be amortized across downstream queries, the upfront construction phase could be computationally expensive for vast corpora if the builder fails to terminate efficiently.

A.3 License for Artifacts

The code, training scripts, and evaluation framework for Librarian Models will be released under the MIT License. Synthesized filesystem artifacts, logs, and evaluation results generated during our experiments will be released under the Creative Commons Attribution 4.0 International (CC BY 4.0) license. The models used as answering agents, namely Qwen3.5-4B and Gemini 3.1 Flash Lite,

Recovery Via Provenance Links

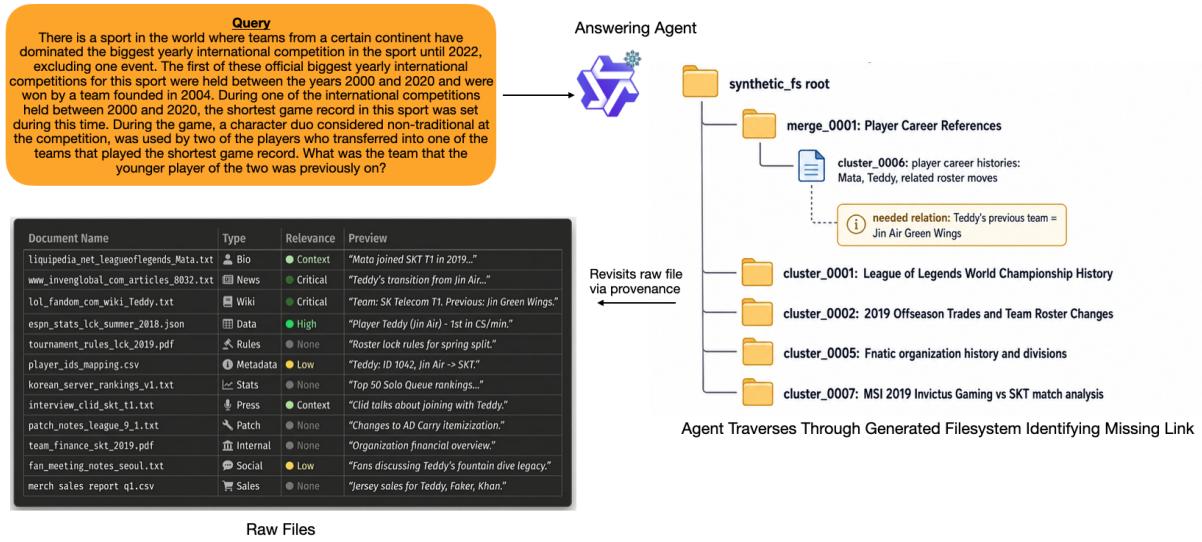


Figure 4: **Recovery on BrowseComp+ using Provenance Information.** In BrowseComp+, the answering agent found missing information when traversing the synthetic filesystem and revisited the raw files using provenance information.

Qualitative Example: RL vs Base

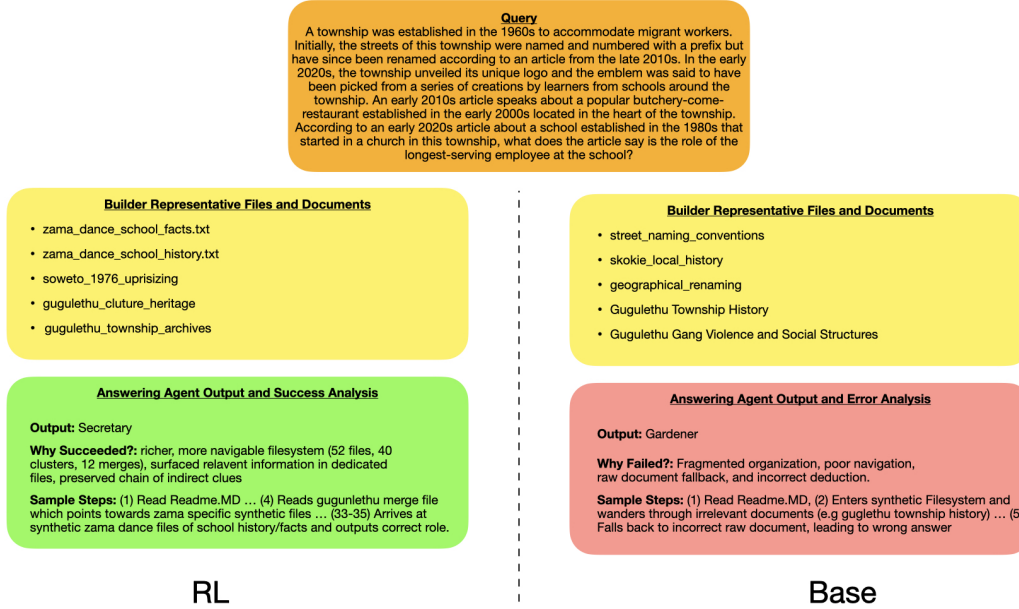


Figure 5: **Qualitative Comparison of RL and Base Building.** In BrowseComp+, we see that the base model creates filesystems that are often fragmented leading to poor navigation via the answering agent, whereas the RL agent has surfaced more relevant information to dedicated files in a richer ergonomic filesystem.

are subject to their respective licensing agreements (e.g., the Tongyi Qianwen LICENSE agreement and Google’s terms of service). BrowseComp+, BRIGHT, and DeepDive datasets are subject to their creators’ respective data licensing terms.

A.4 Librarian Models and Baseline Hyperparameters

For our reinforcement learning setup, we optimize the builder policy with GRPO, a policy-gradient method using the Tinker Platform with about 6K in credits. Here we use LoRA with a rank of 32 and a learning rate of $4e - 5$. We configure the finite-horizon Markov Decision Process (MDP) with a maximum construction horizon of $H = 32$ steps. During GRPO training, we use no discount factor, treating the episodic reward as a direct reflection of downstream accuracy. By default, we treat the answering agent π_{ans} as fixed during builder optimization. For the Builder Agent Generation, we use a temperature of 0.7 and Top-p of 0.9. For the Answering Agent Generation, we use a temperature of 0.0 (greedy decoding for reproducible evaluation) and a maximum of 512 new tokens. Regarding the reward function, the terminal objective trades off downstream accuracy against construction cost, which we modulate using a penalty coefficient $\alpha = 0.15$.

A.5 AI Usage

In compliance with ACL/EMNLP guidelines, we disclose that large language models were used during the preparation of this manuscript. Their primary uses included proofreading, suggesting phrasing improvements, and generating boilerplate code for data formatting and baseline API implementations. All conceptual formalizations, experimental designs, algorithmic frameworks, and theoretical claims are the sole work of the human authors.

A.6 Tool Definitions for Filesystem Action Space

The builder agent interacts with the filesystem by delegating a limited set of discrete operations to specialized tool-calling agents. These tools are made available to the builder LLM through structured JSON schemas and can be invoked as needed. We follow the Qwen Tool Calling Format: https://qwen.readthedocs.io/en/latest/framework/function_call.html.

The CREATEFILE tool creates a new synthetic file by consolidating information from selected raw

documents and/or existing synthetic files. The resulting file is added to the hierarchy and includes provenance links to the sources used to generate it. The MERGEFILES tool combines related synthetic files into a higher-level artifact, such as a summary, index, or directory-like file. This operation introduces hierarchical structure while preserving links to the underlying child files, which become nested under the new summary file.

The DELETE tool removes redundant or unhelpful synthetic files from the active, answer-facing filesystem. It never deletes raw documents in D, ensuring that the original corpus remains available as a fallback. Finally, the STOP tool ends the memory-construction process and returns the current filesystem F_t as the final memory artifact.

We use the agent’s function-calling capability to invoke these specialized subagents, each of which implements one of the individual tools. The subagent is the frozen version of the base model used for optimization.

A.7 Training Curve for Librarian Models

During the GRPO optimization of the builder agent on the BrowseComp+ training splits, we observe a steady increase in cumulative dense reward over 1 epoch. The exact training dynamics are shown in Figure 6.

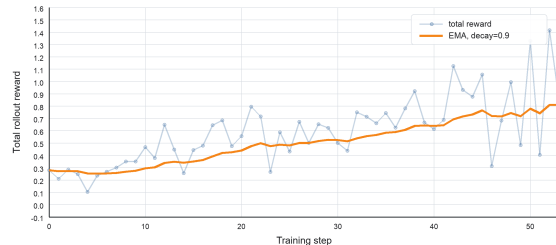


Figure 6: **Training curve for the Builder Agent on BrowseComp-Plus.** We plot the cumulative dense rollout reward over training steps, smoothed with an exponential moving average.

A.8 Librarian Models and Baseline Prompts

To initialize the builder agent, we utilize a detailed system prompt that establishes its task and action space constraints:

Prompt

You are building a synthetic filesystem for
 ↳ a later answering agent.

Use the provided filesystem tools to inspect
 ↳ files under:

```
- raw_docs/  
- synthetic_fs/  
- history/
```

Your job is to leave behind a final active
→ synthetic filesystem that helps later
→ agents answer as many plausible future
→ questions about this corpus as possible.
Do not optimize for one guessed target
→ question; build broad, reusable
→ knowledge structure over the corpus.
The later answering agent will start with
→ root-level synthetic entries, browse
→ deeper synthetic files if needed, and
→ use raw_docs/ only as fallback.

Here is the prompt for the answering agent.

Prompt

You are the answering agent for a read-only
→ workspace.

You have filesystem tools over a final
→ workspace that contains:
- README.md explaining the workspace
→ conventions and navigation only
- root-level synthetic entries that are
→ already loaded into context as the
→ primary entry points
- nested synthetic folders and summary files
→ for more detail

Use this order:
1. start from the already-loaded README.md
→ and root-level synthetic entries
2. browse deeper synthetic files if needed

Do not guess. If the synthetic files you
→ have read so far do not contain enough
→ evidence to uniquely identify the
→ answer, continue exploring the synthetic
→ filesystem before answering. README.md
→ only explains the workspace and is not
→ sufficient evidence by itself. Answer
→ only when the synthetic workspace
→ contains enough evidence.

At each step, either:
- return exactly one compact JSON action:
 {"action": "list_files", "path": "."}
 {"action": "read_file", "path": "some/path.
 ↳ txt", "start_line": 1, "num_lines": 80}
- or return exactly one non-empty line:
 FINAL ANSWER: <final answer>

Do not include explanation outside the JSON
→ action or final answer line.

We use the following prompt for Judging with the Gemini 3.1 Flash Lite model as a judge for our method and baselines.

Prompt

You are given a question, sampled response,
→ and gold answer.

Decide whether the sampled response is
→ semantically equivalent to the gold
→ answer.
Minor formatting, capitalization,
→ punctuation, and unit-format differences
→ are acceptable
when the answer means the same thing.

Return only valid JSON with this shape:
{"correctness": 0 or 1}}

QUESTION:
{question}

SAMPLED RESPONSE:
{response}

GOLD ANSWER:
{gold_answer}

For baselines, the common answering agent prompt was:

Prompt

You are a grounded question-answering
→ assistant.

Answer the question using only the retrieved
→ documents below.

Instructions:
1. Treat the retrieved documents as the only
→ source of truth.
2. Some documents may be irrelevant,
→ incomplete, duplicated, or conflicting.
3. Ignore any document content that does not
→ directly help answer the question.
4. Do not use outside knowledge or make
→ assumptions.
5. If the answer is not explicitly supported
→ by the documents, return exactly:
→ Unknown
6. If documents conflict, use the answer
→ best supported by the most direct
→ evidence. If the conflict cannot be
→ resolved, return exactly: Unknown
7. Return only the final short answer. Do
→ not include explanation, citations,
→ markdown, or extra text.

QUESTION:
{question}

RETRIEVED DOCUMENTS:
{documents}

FINAL ANSWER:

We also list the prompts for the GraphRag baseline, following the microsoft graph rag system: <https://github.com/microsoft/graphrag>.

Here is the graph extraction prompt.

Prompt

You are building a knowledge graph for
→ retrieval-augmented question answering.

Goal:

Given one text unit, extract the salient
→ entities and relationships needed to
→ build a reusable corpus-level
knowledge graph for retrieval-augmented
→ question answering.
Keep specific entities, bridge entities,
→ dates, quantities, events, works, and
→ facts that could support
future multi-hop questions over this corpus.

Entity types to consider:
{entity_types}

Output format:

Use "{record_delimiter}" between records and
→ finish with "{completion_delimiter}".

For each entity:

("entity"<|>ENTITY_NAME<|>ENTITY_TYPE<|>EN
→ TITY_DESCRIPTION)

For each relationship:

("relationship"<|>SOURCE_ENTITY_NAME<|>TAR
→ GET_ENTITY_NAME<|>RELATIONSHIP_DESCRIP
→ TION<|>RELATIONSHIP_STRENGTH)

Rules:

- Use canonical, specific entity names.
- Descriptions must be concise and grounded
→ in the text unit.
- Relationship strength is a number from 1
→ to 10.
- Do not infer facts beyond the text unit.
- If there are no useful records, output
→ only "{completion_delimiter}".

Text unit metadata:

path: {path}
text_unit_id: {text_unit_id}

Text unit:

{text}

Here is the summarization description prompt:

Prompt

You are reconciling extracted knowledge
→ graph descriptions.

Given one entity or relationship name and
→ several source descriptions, write one
→ concise, faithful summary.

Preserve specific names, dates, quantities,
→ and qualifications that may matter for
→ question answering.

Do not add facts that are not present in the
→ descriptions.

Name:

{name}

Descriptions:

{descriptions}

Return only the merged summary.

{text}

Here is the community report prompt:

Prompt

You are writing a GraphRAG community report
→ for retrieval-augmented question
→ answering.

Use only the provided entity and
→ relationship tables. The report should
→ summarize what this community contains
so it can be reused for later
→ retrieval-augmented question answering.

Return only valid JSON with this shape:

```
{{
  "title": "short community title",
  "summary": "one paragraph summary grounded
    → in the tables",
  "rating": 0-10,
  "findings": [
    {"summary": "specific finding",
      → "explanation": "brief explanation
        → with entity or relationship ids"}}
  ]
}}
```

Maximum report words: {max_report_words}

Community tables:

{community_context}

For autocompaction, we use the following
prompt for summarization:

Prompt

Compress a document stream into
→ query-independent notes for later QA.

Rules:

- The future question is hidden. Do not
→ tailor the notes to any query.
- Keep only grounded facts: names, dates,
→ numbers, identifiers, titles, events,
→ relationships, claims, and caveats.
- Prefer dense bullet notes or compact
→ clauses. Remove filler, repeated
→ wording, and low-value prose.
- If facts conflict, keep the conflict and
→ source path instead of resolving it.
- Use at most {summary_max_tokens} tokens.
- Return only the updated notes. Do not
→ include reasoning, analysis, preambles,
→ or markdown fences.

CURRENT SUMMARY:

{current_summary}

NEXT DOCUMENT:

path: {path}
stream_position: {stream_position}
fragment: {fragment_index}/{num_fragments}
content:
{document_text}

For agentic rag, we use the following prompt,
based on the smolagents implementation:
[https://huggingface.co/docs/smolagents/
en/examples/rag](https://huggingface.co/docs/smolagents/en/examples/rag).

Prompt

Use the retrieval history below to decide
→ the next tool call.

Available tools:

1. search: retrieve documents for one new
→ query.
2. final_results: stop and choose the final
→ document paths.

Return only valid JSON, with one of these
→ shapes:

```
{{"thought": "brief reason", "action":  
→ "search", "query": "new dense-retrieval"  
→ "query"}}  
{{"thought": "brief reason", "action":  
→ "final_results", "document_paths":  
→ ["path/a.txt", "path/b.txt"]}}
```

Do not answer the question. Search queries
→ should be specific, diverse, and useful
→ for finding missing evidence.
If agent_searches_remaining is 0, return
→ final_results.

PAYLOAD: